

LAPORAN PRAKTIKUM
JOB SHEET 6 - ASINKRON KOMUNIKASI FETCH API DAN JSON



Disusun Oleh:

Nama : Mohammad Hafidz Rafi' Rabbani
NIM : 254107020084
Kelas : 2H
No. Absen : 14
Repository : github.com/hafidzrafi/DnPWeb2026

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Daftar Isi

1. Tujuan	3
2. Praktikum	3
2.1. Percobaan 1: Pengelolaan Status Pemuatan (Loading State)	3
2.1.1. 1.1 Perancangan Elemen Loading Indicator	3
2.1.2. 1.2 Signifikansi Indikator bagi Pengalaman Pengguna (UX)	4
2.1.3. Langkah-langkah	4
2.1.4. Pertanyaan	6
2.2. Percobaan 2: Pengambilan dan Perenderan Dinamis Data Buku	7
2.2.1. 2.1 Alur Kerja Fetch API dan JSON Parsing	7
2.2.2. Langkah-langkah	8
2.2.3. Pertanyaan	9
2.3. Percobaan 3: Pengambilan dan Perenderan Dinamis Data Anggota	10
2.3.1. 3.1 Konsistensi Pola Pemrograman Asinkron	10
2.3.2. Langkah-langkah	11
2.3.3. Pertanyaan	12
2.4. Percobaan 4: Penerapan Event Delegation pada Elemen Dinamis	13
2.4.1. 4.1 Kegagalan Pendekatan QuerySelector Konvensional	13
2.4.2. 4.2 Solusi Event Delegation Mengandalkan Event Bubbling	13
2.4.3. Langkah-langkah	14
2.4.4. Pertanyaan	15
2.5. Percobaan 5: Penanganan Kesalahan Komunikasi Jaringan (Error Handling) . . .	16
2.5.1. 5.1 Isolasi Kegagalan dengan try...catch	16
2.5.2. 5.2 Penyajian Pesan Kesalahan Terpadu	16
2.5.3. Langkah-langkah	17
2.5.4. Pertanyaan	18
3. Tugas	19
3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 6	19

1. Tujuan

- Memahami konsep dasar arsitektur komunikasi asinkron berbasis AJAX (Asynchronous JavaScript and XML) dan format serialisasi data JSON (JavaScript Object Notation).
- Menguasai pemanggilan antarmuka pemrograman jaringan modern menggunakan Fetch API berbasis Promise dan sintaksis `async / await`.
- Mengimplementasikan pemisahan lapisan data (data layer) dari struktur dokumen presentasi dengan mengosongkan tag `<tbody>` pada dokumen HTML.
- Membangun mekanisme umpan balik visual status pemuatan antarmuka (loading indicator) selama latensi transfer jaringan berlangsung.
- Menerapkan penanganan kegagalan tangguh (resilient error handling) memanfaatkan blok `try...catch...finally` guna mencegah kegagalan fatal pada antarmuka pengguna.
- Menguasai teknik Event Delegation pada objek tingkat tinggi dokumen (document) untuk menangani elemen antarmuka yang terkonstruksi secara dinamis pasca-DOMContentLoaded.
- Menganalisis batasan keamanan peramban modern terkait kebijakan Same-Origin Policy (SOP) dan Cross-Origin Resource Sharing (CORS) saat berinteraksi dengan berkas lokal.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: <https://github.com/hafidzrafi/DnPWeb2026>.

2. Praktikum

2.1. Percobaan 1: Pengelolaan Status Pemuatan (Loading State)

Pada percobaan pertama, antarmuka daftar buku dan anggota dipersiapkan untuk menghadapi latensi komunikasi jaringan asinkron. Berbeda dengan halaman web statis di mana seluruh konten langsung tersedia saat file dimuat, operasi jaringan membutuhkan waktu bervariasi tergantung kondisi koneksi.

2.1.1. 1.1 Perancangan Elemen Loading Indicator

Sebuah paragraf penanda status `#loading-indicator` disisipkan di atas tabel dengan gaya awal tersembunyi (`display: none`):

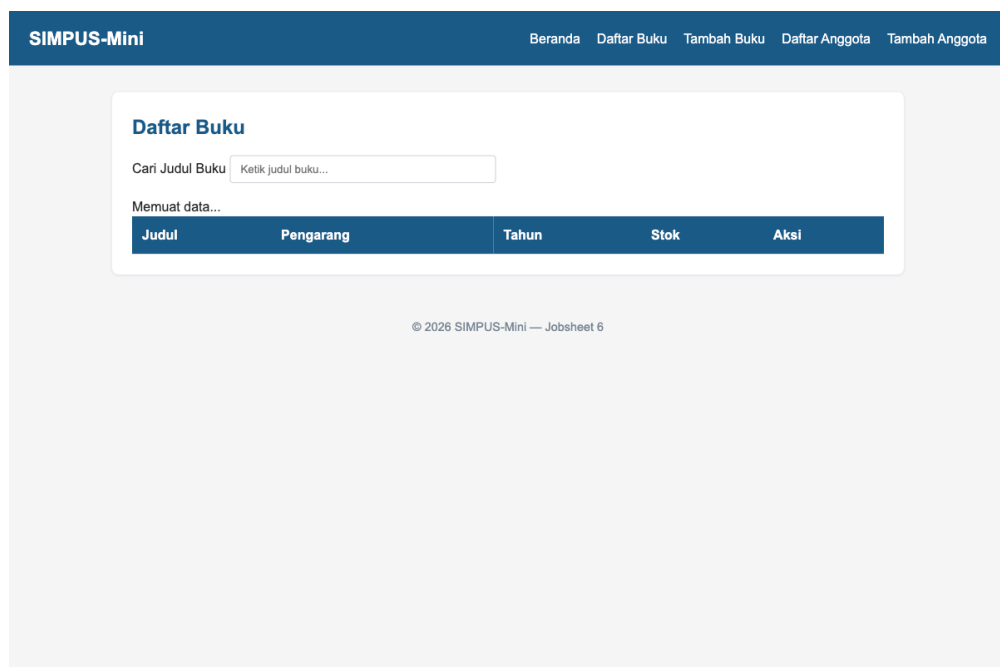
```
<p id="loading-indicator" style="display:none;">Memuat data...</p>
```

Di dalam fungsi pengendali data, indikator ini langsung diaktifkan dengan mengubah properti tampilan menjadi `loading.style.display = "block"` tepat sebelum permintaan jaringan `fetch()` dieksekusi. Untuk mensimulasikan jeda jaringan nyata pada pengujian lokal, digunakan penundaan buatan selama 600 milidetik menggunakan `Promise`:

```
await new Promise((resolve) => setTimeout(resolve, 600));
```

2.1.2. 1.2 Signifikansi Indikator bagi Pengalaman Pengguna (UX)

Tanpa adanya umpan balik visual, pengguna dapat mengasumsikan bahwa aplikasi membeku (**frozen**) atau tabel dalam kondisi kosong permanen. Teks “Memuat data...” memberikan kepastian bahwa sistem sedang aktif memproses permintaan di latar belakang.



Gambar 1.1: Indikator Status “Memuat data...” Tampil Sesaat Selagi Proses Asinkron Berlangsung.

2.1.3. Langkah-langkah

1. Menambahkan elemen `<p id="loading-indicator" style="display:none;">Memuat data...</p>` pada `books/list.html` dan `members/list.html`.
2. Mengosongkan seluruh isi tag `<tbody>` pada kedua tabel HTML.
3. Menambahkan logika aktivasi indikator pada blok awal fungsi `muatDaftarBuku()` dan menonaktifkan pada blok `finally`.

4. Memverifikasi kemunculan teks loading saat halaman pertama kali dibuka sebelum data dirender.

2.1.4. Pertanyaan

1. Apa peran vital dari blok `finally` dalam penanganan indikator pemuatan?
 - Blok `finally` menjamin operasi pembersihan (**cleanup operations**) pasti dieksekusi dalam kondisi apa pun, baik permintaan jaringan berhasil tuntas di blok `try` maupun terhenti akibat kegagalan koneksi di blok `catch`. Hal ini mencegah indikator loading tergantung atau berputar selamanya di layar antarmuka pengguna ketika terjadi error.

2.2. Percobaan 2: Pengambilan dan Perenderan Dinamis Data Buku

Percobaan kedua merupakan inti dari arsitektur **Client-Side Rendering (CSR)** pada Jobsheet 6. Seluruh baris data buku yang sebelumnya ditulis secara manual di HTML kini dimuat secara asinkron dari berkas terstruktur `data/buku.json`.

2.2.1. 2.1 Alur Kerja Fetch API dan JSON Parsing

Fungsi `muatDaftarBuku()` pada berkas `assets/js/buku.js` mengeksekusi urutan operasi berikut:

```
async function muatDaftarBuku() {
  const tbody = document.querySelector(".table-responsive table tbody");
  const loading = document.getElementById("loading-indicator");
  if (!tbody) return;

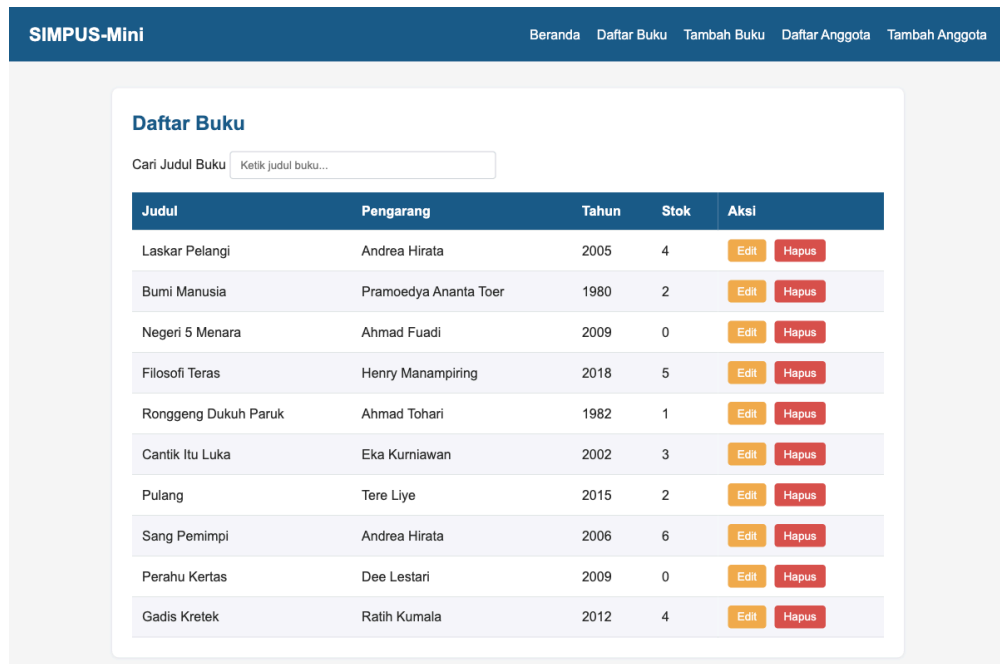
  loading.style.display = "block";
  tbody.innerHTML = "";

  try {
    await new Promise((resolve) => setTimeout(resolve, 600));

    const res = await fetch("../data/buku.json");
    if (!res.ok) {
      throw new Error("Gagal mengambil data (status " + res.status + ")");
    }
    const daftarBuku = await res.json();

    daftarBuku.forEach(function (buku) {
      const tr = document.createElement("tr");
      tr.innerHTML =
        "<td>" + buku.judul + "</td>" +
        "<td>" + buku.pengarang + "</td>" +
        "<td>" + buku.tahun + "</td>" +
        "<td>" + buku.stok + "</td>" +
        "<td>" +
        "<button type='button'>Edit</button> " +
        "<button type='button' class='btn-hapus'>Hapus</button>" +
        "</td>";
      tbody.appendChild(tr);
    });
  } catch (err) {
    tbody.innerHTML =
      "<tr><td colspan='5'>Gagal memuat data: " + err.message + "</td></tr>";
  } finally {
    loading.style.display = "none";
  }
}
```

Method `fetch()` mengembalikan objek `Response`. Pengecekan manual `res.ok` memvalidasi bahwa kode status HTTP berada pada rentang 200–299. Selanjutnya, method `res.json()` mem-parse string format JSON menjadi larik (**array**) objek JavaScript asli. Setiap objek buku diiterasi menggunakan `forEach()` untuk membangun elemen baris `<tr>` baru dan dimasukkan ke dalam `<tbody>` memanfaatkan `tbody.appendChild(tr)`.



Judul	Pengarang	Tahun	Stok	Aksi
Laskar Pelangi	Andrea Hirata	2005	4	Edit Hapus
Bumi Manusia	Pramoedya Ananta Toer	1980	2	Edit Hapus
Negeri 5 Menara	Ahmad Fuadi	2009	0	Edit Hapus
Filosofi Teras	Henry Manampiring	2018	5	Edit Hapus
Ronggeng Dukuh Paruk	Ahmad Tohari	1982	1	Edit Hapus
Cantik Itu Luka	Eka Kurniawan	2002	3	Edit Hapus
Pulang	Tere Liye	2015	2	Edit Hapus
Sang Pemimpi	Andrea Hirata	2006	6	Edit Hapus
Perahu Kertas	Dee Lestari	2009	0	Edit Hapus
Gadis Kretek	Ratih Kumala	2012	4	Edit Hapus

Gambar 1.2: Hasil Konsumsi Asinkron `data/buku.json` yang Dirender Menjadi 10 Baris Tabel Dinamis.

2.2.2. Langkah-langkah

1. Menyusun berkas `data/buku.json` berisi 10 rekaman objek buku lengkap dengan atribut judul, pengarang, tahun, dan stok.
2. Mengimplementasikan fungsi `muatDaftarBuku()` pada berkas `assets/js/buku.js`.
3. Memasang pemanggilan fungsi otomatis saat event `'DOMContentLoaded'` terpicu.
4. Memverifikasi melalui browser bahwa 10 data buku terrender sempurna ke dalam tabel tanpa pemuatan ulang halaman.

2.2.3. Pertanyaan

1. Mengapa properti `res.ok` harus diperiksa secara manual sebelum menjalankan `res.json()`?
 - Karena Fetch API dirancang hanya me-reject Promise jika terjadi kegagalan jaringan fatal tingkat transport (seperti jaringan offline atau DNS gagal diresolusi). Apabila server mengembalikan status error HTTP seperti 404 (Not Found) atau 500 (Internal Server Error), Fetch API tetap menganggapnya sebagai resolusi berhasil. Oleh karena itu, jika `!res.ok` tidak ditangkap secara eksplisit, eksekusi `res.json()` terhadap payload halaman HTML error 404 akan memicu **SyntaxError: Unexpected token**.

2.3. Percobaan 3: Pengambilan dan Perenderan Dinamis Data Anggota

Percobaan ketiga menerapkan arsitektur serupa pada modul anggota perpustakaan menggunakan berkas data `data/anggota.json` dan skrip pengendali `assets/js/anggota.js`.

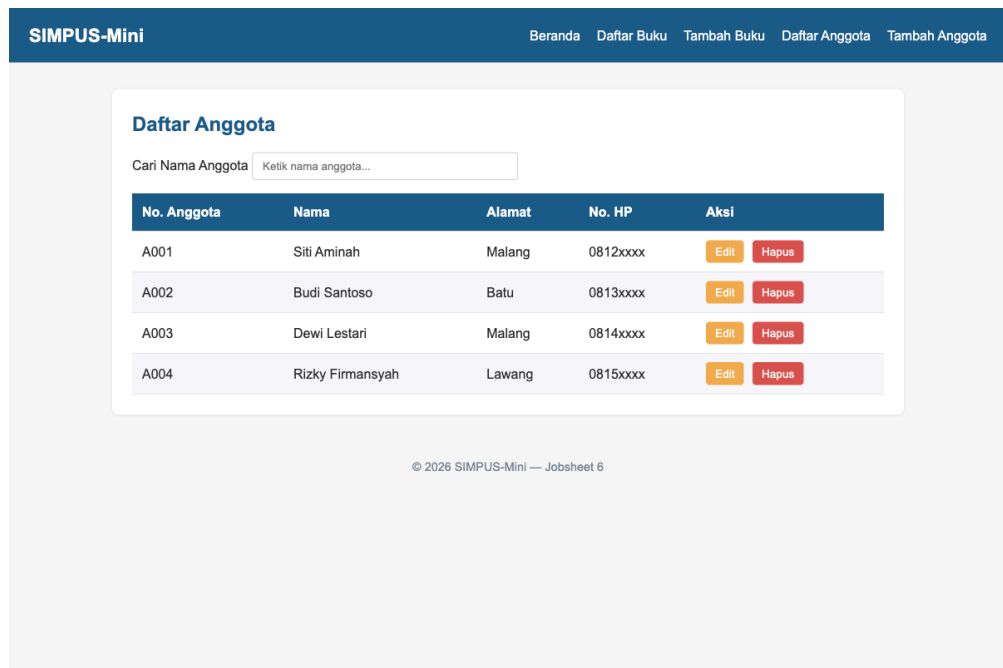
2.3.1. 3.1 Konsistensi Pola Pemrograman Asinkron

Fungsi `muatDaftarAnggota()` membaca data 4 orang anggota perpustakaan yang mencakup atribut nomor anggota (`no_anggota`), nama lengkap, alamat tempat tinggal, dan nomor telepon seluler:

```
async function muatDaftarAnggota() {
  // ...inisialisasi loading & pembersihan tbody...
  const res = await fetch("../data/anggota.json");
  if (!res.ok) throw new Error("Gagal mengambil data (status " + res.status + ")");
  const daftarAnggota = await res.json();

  daftarAnggota.forEach(function (anggota) {
    const tr = document.createElement("tr");
    tr.innerHTML =
      "<td>" + anggota.no_anggota + "</td>" +
      "<td>" + anggota.nama + "</td>" +
      "<td>" + anggota.alamat + "</td>" +
      "<td>" + anggota.no_hp + "</td>" +
      "<td>" +
      "<button type='button'>Edit</button> " +
      "<button type='button' class='btn-hapus'>Hapus</button>" +
      "</td>";
    tbody.appendChild(tr);
  });
}
```

Penerapan pola ini membuktikan fleksibilitas konsep perenderan dinamis sisi klien, di mana struktur logika kode yang seragam dapat menyajikan berbagai skema data berbeda hanya dengan menyesuaikan struktur JSON sumbernya.



Gambar 1.3: Tabel Anggota Perpustakaan Berhasil Memuat 4 Rekaman Data Secara Dinamis dari `data/anggota.json`.

2.3.2. Langkah-langkah

1. Menyusun berkas ``data/anggota.json`` dengan data anggota perpustakaan.
2. Mengembangkan skrip ``assets/js/anggota.js`` dan menautkannya pada ``members/list.html``.
3. Menguji perenderan tabel anggota melalui peramban web dan memeriksa struktur elemen melalui tab Elements DevTools.

2.3.3. Pertanyaan

1. Bagaimana struktur arsitektur ini mempermudah transisi menuju integrasi RESTful API di masa mendatang?
 - Pada struktur ini, kode antarmuka JavaScript telah sepenuhnya terisolasi dari format penyimpanan data. Di masa depan, pengembang hanya perlu mengubah URL target pemanggilan `fetch("../data/anggota.json")` menjadi endpoint API backend sungguhan (misalnya `fetch("/api/anggota")`) tanpa perlu merombak logika perenderan DOM dan tata letak tabel antarmuka pengguna.

2.4. Percobaan 4: Penerapan Event Delegation pada Elemen Dinamis

Percobaan keempat mengatasi masalah fundamental dalam pemrograman berbasis kejadian (**event-driven programming**) saat berhadapan dengan elemen yang dibuat secara dinamis setelah dokumen selesai dimuat.

2.4.1. 4.1 Kegagalan Pendekatan QuerySelector Konvensional

Pada Jobsheet 5, tombol hapus dipasang penangan event menggunakan:

```
document.querySelectorAll(".btn-hapus").forEach(...)
```

Pendekatan di atas mengalami kegagalan total (**silent failure**) pada Jobsheet 6. Karena baris tabel dibuat secara asinkron beberapa ratus milidetik setelah inisialisasi awal, pemanggilan `querySelectorAll` saat `DOMContentLoaded` hanya menemukan elemen kosong (`NodeList(0)`). Akibatnya, tombol-tombol Hapus yang baru muncul tidak memiliki listener terpasang.

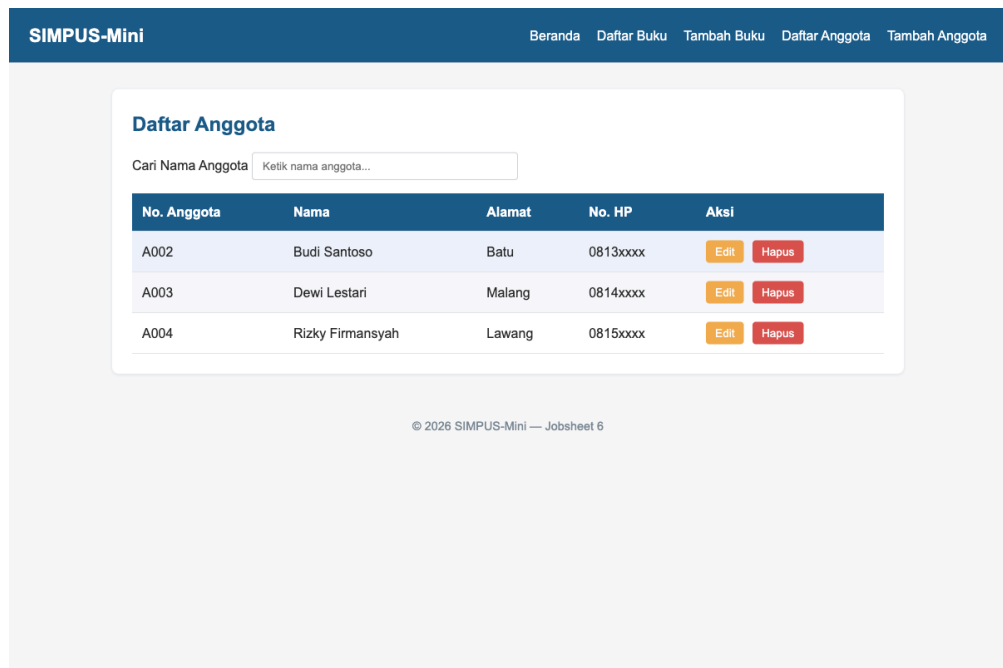
2.4.2. 4.2 Solusi Event Delegation Mengandalkan Event Bubbling

Solusi rekayasa yang tepat adalah **Event Delegation**, yaitu mendaftarkan satu-satunya event listener pada objek tingkat teratas dokumen (`document`):

```
function initHapusConfirm() {
  document.addEventListener("click", function (e) {
    const btn = e.target.closest(".btn-hapus");
    if (!btn) return;

    const row = btn.closest("tr");
    const nama = row ? row.querySelector("td").textContent : "data ini";
    const yakin = confirm("Yakin ingin menghapus \"" + nama + "\"?");
    if (yakin && row) {
      row.remove();
    }
  });
}
```

Ketika pengguna mengklik elemen apa pun di layar, event klik akan merambat naik menuju root dokumen (**Event Bubbling**). Method `e.target.closest(".btn-hapus")` menyaring kejadian tersebut: jika klik tidak berasal dari tombol hapus atau turunannya, fungsi langsung mengembalikan nilai (**early return**). Jika cocok, dialog konfirmasi diaktifkan dan baris target dicabut dari DOM.



Gambar 1.4: Baris Data Anggota Pertama (A001) Berhasil Dihapus Menggunakan Pendekatan Event Delegation.

2.4.3. Langkah-langkah

1. Memodifikasi implementasi fungsi ``initHapusConfirm()`` di dalam ``assets/js/app.js`` menggunakan ``document.addEventListener("click", ...)``.
2. Menggunakan ``e.target.closest(".btn-hapus")`` untuk mendeteksi target interaksi.
3. Menguji klik tombol Hapus pada baris buku dan anggota yang dibuat secara dinamis.
4. Memverifikasi bahwa dialog konfirmasi tetap muncul dan baris yang dipilih berhasil terhapus dari DOM.

2.4.4. Pertanyaan

1. Sebutkan dua keuntungan utama penggunaan Event Delegation dari sudut pandang efisiensi memori!
 - 1. **Optimalisasi Penggunaan Memori:** Mencegah pembuatan ratusan objek listener fungsi di memori peramban. Berapa pun jumlah baris yang ada di dalam tabel (bahkan ribuan), peramban hanya menyimpan satu buah listener di level dokumen.
 - 2. **Ketahanan Elemen Dinamis:** Setiap elemen baru yang ditambahkan ke dalam DOM di kemudian hari secara otomatis langsung dapat berinteraksi tanpa memerlukan registrasi listener ulang.

2.5. Percobaan 5: Penanganan Kesalahan Komunikasi Jaringan (Error Handling)

Percobaan kelima menguji ketahanan aplikasi saat menghadapi kegagalan jaringan atau berkas sumber data yang tidak ditemukan (**graceful degradation**).

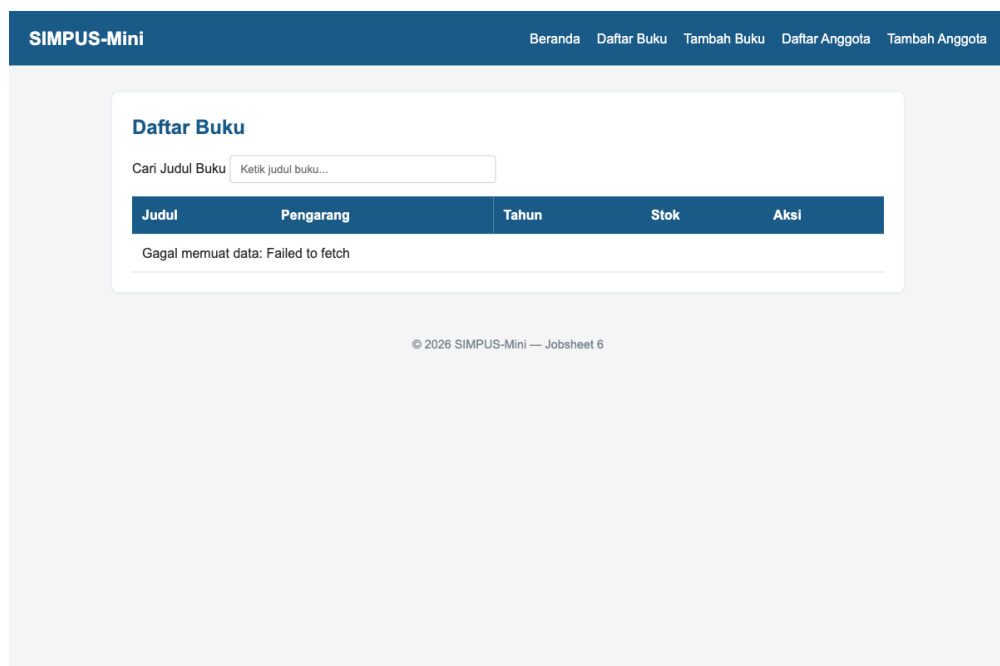
2.5.1. 5.1 Isolasi Kegagalan dengan try...catch

Blok `try...catch` disematkan untuk memayungi seluruh operasi kritis. Apabila permintaan `fetch()` mengalami kegagalan (misalnya koneksi server offline atau nama berkas salah), alur program tidak terhenti, melainkan langsung ditangkap oleh blok `catch (err)`:

```
try {
  // ...operasi fetch...
} catch (err) {
  tbody.innerHTML =
    "<tr><td colspan='5'>Gagal memuat data: " + err.message + "</td></tr>";
} finally {
  loading.style.display = "none";
}
```

2.5.2. 5.2 Penyajian Pesan Kesalahan Terpadu

Daripada membiarkan halaman rusak atau menampilkan konsol error merah tanpa informasi visual, aplikasi secara cerdas mengganti konten `<tbody>` dengan sebuah baris tunggal berspan 5 kolom yang menginformasikan rincian galat kepada pengguna.



Gambar 1.5: Pesan Kesalahan Ditampilkan Elegan di Dalam Tabel Saat Terjadi Kegagalan Jaringan.

2.5.3. Langkah-langkah

1. Mengimplementasikan blok penanganan ``try...catch...finally`` pada fungsi pembacaan data.
2. Menguji skenario kegagalan jaringan (misal mengubah sementara path file target fetch atau menghentikan server).
3. Mengamati tabel menampilkan pesan error yang ramah pengguna dan indikator loading tertutup rapi.

2.5.4. Pertanyaan

1. Mengapa aplikasi web profesional tidak boleh membiarkan error jaringan hanya tercatat di Console browser?
 - Pengguna akhir (**end-users**) aplikasi web tidak membuka tab Developer Tools Console. Jika error jaringan hanya dicatat di konsol, pengguna akan melihat antarmuka kosong yang membingungkan dan mengira aplikasi mengalami **hang**. Menampilkan pesan kesalahan informatif di antarmuka memberi tahu pengguna situasi yang sebenarnya terjadi dan langkah yang perlu diambil (misalnya memeriksa koneksi internet atau memuat ulang halaman).

3. Tugas

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 6

Berikut adalah evaluasi kritis dan analisis mendalam terhadap lima pokok pertanyaan konseptual pada Jobsheet 6:

No	Pertanyaan dan Jawaban Konseptual
1	Soal: Apa keuntungan utama arsitektur komunikasi data asinkron (asynchronous communication) menggunakan Fetch API dibandingkan pemuatan ulang halaman web secara menyeluruh (full page reload)? Jawaban: 1. Efisiensi Beban Jaringan (Payload Bandwidth): Pada pemuatan ulang konvensional, server harus mentransmisikan ulang seluruh kerangka dokumen HTML, stylesheet CSS, skrip JavaScript, serta aset media. Dengan Fetch API asinkron, transfer data hanya terbatas pada muatan data mentah terstruktur (format JSON) berukuran sangat ringkas, menghemat bandwidth secara masif. 2. Kontinuitas Pengalaman Pengguna (Smooth UX): Layar peramban tidak mengalami fenomena kedipan putih (white flash) maupun kehilangan status scroll dan posisi fokus saat data diperbarui. 3. Pemisahan Tugas Arsitektur: Mengukuhkan arsitektur modern di mana server web murni berperan sebagai penyedia data (API Provider), sementara logika presentasi grafis didelegasikan ke mesin peramban pengguna (Client-Side Rendering).
2	Soal: Mengapa eksekusi fungsi <code>fetch()</code> selalu mengembalikan objek Promise dan harus dipanggil menggunakan kata kunci <code>await</code> di dalam fungsi <code>async</code>? Mengapa properti <code>res.ok</code> tetap wajib diverifikasi secara manual sebelum mem-parsing data JSON? Jawaban: Fungsi <code>fetch()</code> mengembalikan objek <code>Promise</code> karena transmisi data jaringan bersifat I/O-bound dan membutuhkan waktu tunggu (latency)

No	Pertanyaan dan Jawaban Konseptual
	yang tidak deterministik. Mengingat mesin JavaScript beroperasi pada single-threaded event loop, proses penantian tersebut harus berjalan secara non-pemblokir (non-blocking). Sintaksis <code>async/await</code> menyediakan abstraksi elegan agar penulisan logika asinkron dapat dibaca secara sekuensial lurus tanpa terjebak dalam perangkat callback hell. Properti <code>res.ok</code> (bernilai <code>true</code> jika status kode HTTP berada di rentang 200–299) wajib diverifikasi manual karena arsitektur bawaan Fetch API hanya akan me-reject Promise jika terjadi kegagalan jaringan fisik (misalnya koneksi putus atau server mati total). Ketika server mengembalikan status galat HTTP 404 (Not Found) atau 500 (Internal Server Error), Fetch API tetap memperlakukannya sebagai operasi sukses (fulfilled Promise). Tanpa pemeriksaan <code>!res.ok</code>, pemanggilan berikutnya <code>res.json()</code> terhadap badan halaman HTML galat 404 akan memicu kesalahan fatal SyntaxError: Unexpected token saat proses parsing dilakukan.
3	Soal: Bagaimana struktur penanganan kegagalan <code>try...catch...finally</code> menjaga aplikasi tetap resilient (tangguh) saat terjadi kegagalan jaringan atau berkas tidak ditemukan? Apa peran spesifik dari blok <code>finally</code> pada kasus loading indicator? Jawaban: Struktur <code>try...catch...finally</code> mengisolasi blok kode rawan kegagalan di dalam <code>try</code>. Ketika kegagalan jaringan terdeteksi atau pengecualian dilempar melalui pernyataan <code>throw new Error(...)</code>, aliran komputasi langsung dialihkan ke blok <code>catch(err)</code> untuk menampilkan komponen pesan kegagalan ramah pengguna di dalam tabel, menjaga agar seluruh sistem aplikasi tetap beroperasi secara anggun tanpa mengalami runtime crash. Peran spesifik dari blok <code>finally</code> adalah menjalankan prosedur pembersihan mutlak (mandatory cleanup). Dalam skenario antarmuka ini, menyembunyikan indikator <code>loading.style.display = "none"</code> ditempatkan di dalam blok <code>finally</code>

No	Pertanyaan dan Jawaban Konseptual
	untuk memastikan bahwa indikator pemuatan pasti disembunyikan kembali tanpa memandang apakah operasi <code>try</code> berakhir dengan keberhasilan data atau kegagalan yang ditangkap oleh <code>catch</code>.
4	Soal: Mengapa pada Jobsheet 6 penanganan event klik tombol Hapus (<code>.btn-hapus</code>) wajib dialihkan menggunakan teknik Event Delegation pada objek leluhur (<code>document</code>) dan tidak bisa lagi menggunakan <code>querySelectorAll(".btn-hapus").forEach(...)</code> seperti pada Jobsheet 5? Jawaban: Pada Jobsheet 5, seluruh baris tabel ditulis secara statis di dokumen HTML sehingga seluruh tombol <code>.btn-hapus</code> telah eksis di memori DOM saat event <code>DOMContentLoaded</code> selesai terpicu. Sebaliknya, pada Jobsheet 6, penampung <code><tbody></code> berada dalam keadaan kosong saat pembacaan skrip dimulai. Elemen-elemen baris tabel baru diinjeksikan ke dalam pohon DOM beberapa saat kemudian setelah penyelesaian request jaringan asinkron. Penggunaan metode <code>querySelectorAll</code> pada tahap awal hanya akan menghasilkan koleksi elemen kosong (<code>NodeList(0)</code>), menyebabkan seluruh tombol Hapus dinamis tidak memiliki penanganan event klik. Teknik Event Delegation menempatkan satu penanganan event tunggal pada node leluhur abadi (<code>document</code>). Mengandalkan fase perambatan kejadian ke atas (Event Bubbling), setiap klik pada area mana pun di layar akan tereskalasi ke tingkat dokumen. Penyeleksian berbasis <code>e.target.closest(".btn-hapus")</code> kemudian menyaring interaksi tersebut secara efisien: mengabaikan klik pada area lain dan mengeksekusi konfirmasi penghapusan saat tombol Hapus dinamis ditekan.
5	Soal: Mengapa pemanggilan fungsi <code>fetch()</code> ke berkas lokal (<code>buku.json</code> / <code>anggota.json</code>) otomatis diblokir oleh peramban ketika dibuka menggunakan protokol berkas lokal (<code>file://</code>), dan mengapa peramban membutuhkan protokol <code>http://</code> dari server lokal?

No	Pertanyaan dan Jawaban Konseptual
	Jawaban: Peramban web modern menerapkan mekanisme pertahanan keamanan fundamental bernama Same-Origin Policy (SOP) dan pembatasan Cross-Origin Resource Sharing (CORS). Ketika halaman web dibuka secara langsung melalui sistem berkas lokal (<code>file://</code>), peramban memperlakukan origin berkas tersebut sebagai opaque / null origin. Untuk mencegah skrip berbahaya menyusup dan membaca berkas lokal sensitif di media penyimpanan komputer pengguna tanpa izin, peramban secara otomatis memblokir seluruh komunikasi I/O asinkron antar berkas pada protokol <code>file://</code>. Dengan menjalankan server web lokal (seperti <code>php -S localhost:8000</code>), seluruh berkas HTML dan berkas JSON disajikan di bawah identitas origin HTTP resmi yang seragam (<code>http://localhost:8000</code>). Keberadaan origin yang sah ini memenuhi kriteria otorisasi keamanan peramban sehingga eksekusi pertukaran data asinkron via Fetch API diizinkan berjalan normal.